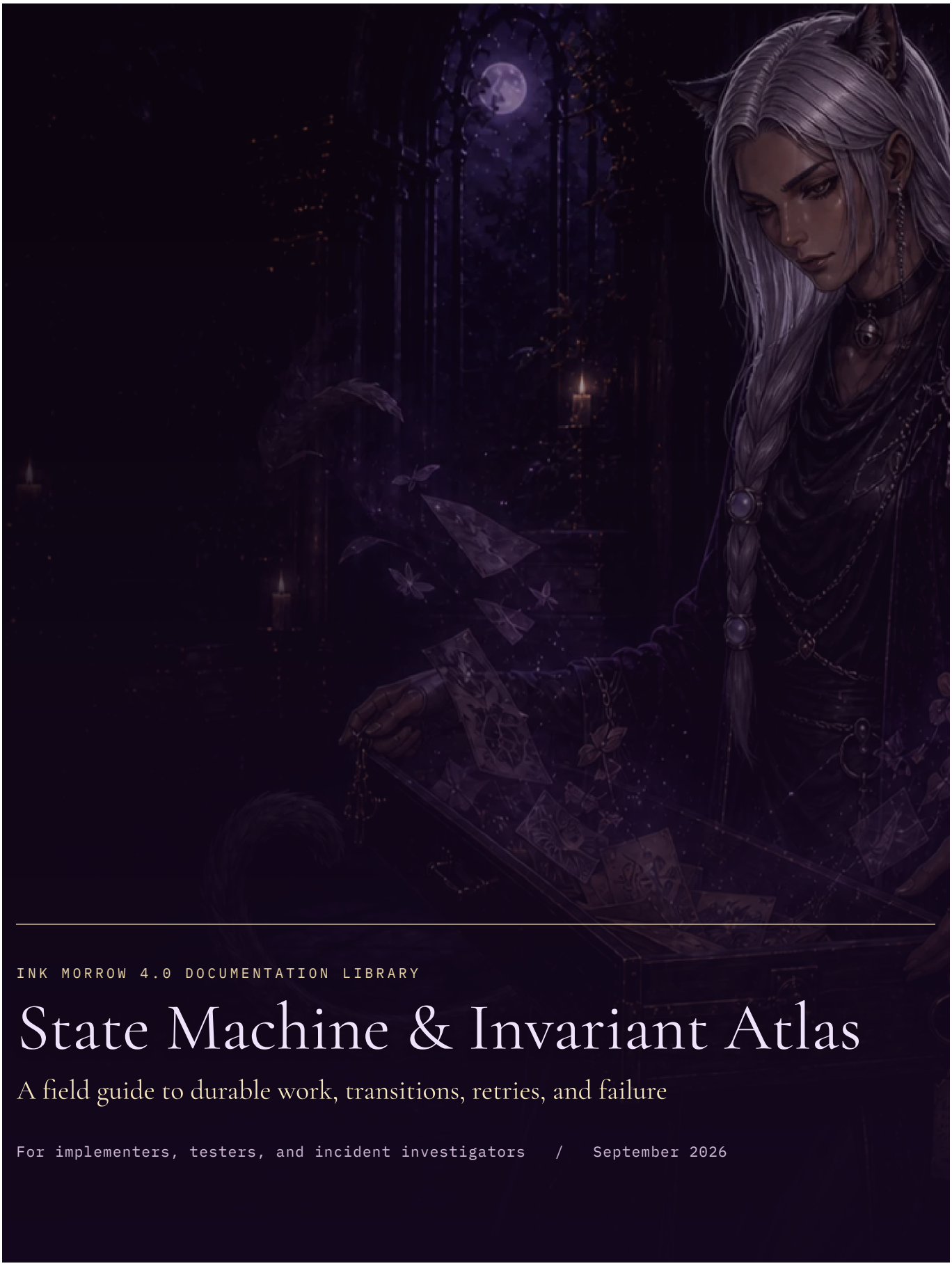


INK MORROW 4.0 DOCUMENTATION LIBRARY

State Machine & Invariant Atlas

A field guide to durable work, transitions, retries, and failure

For implementers, testers, and incident investigators / September 2026

State Machine & Invariant Atlas

Ink Morrow turns uncertain work - networks, providers, browsers, filesystems, and human revision - into durable, inspectable states. This atlas names those states and the invariants between them.

Use it when implementing a transition, designing a test, investigating a stuck badge, planning restart behavior, or deciding whether retry is safe. Database schemas and code remain authoritative; this book is the human map.

Reading notation: `A -> B` is an allowed transition. A guard must be true before transition. An effect is durable work performed inside the boundary. A reconciliation rule explains what happens after restart or a lost response.

Contents

01 Atlas conventions

Every durable machine should answer nine questions:

1. What stable ID names this attempt?
2. What is the initial state and who creates it?
3. Which transitions are allowed, and which actor owns each one?
4. What guard proves the transition still belongs to current context?
5. What database and filesystem effects are atomic?
6. Can provider spend occur, and where is known usage recorded?
7. Is retry a join, a new attempt, or forbidden?
8. What does restart reconciliation do?
9. What exact state and error can the user see?

States should be monotonic unless the model explicitly defines a new attempt. Do not turn `failed` back into `running`; create or join a retry with traceable identity.

02 Writing operation

```
requested -> running -> succeeded -> committed
                |          |
                v          +-> superseded
              failed
```

State	Meaning	Visible consequence
`requested`	Request, idempotency key, expected tail/revision exist	Action accepted but no provider owner yet
`running`	Exactly one provider call owns the attempt	Progress and cancellation affordance
`succeeded`	Complete result and known usage recorded	Result is available but not yet canon
`committed`	Canon transaction consumed the result	Page/revision advances exactly once
`failed`	No canon change; error and known spend recorded	Retry/repair with specific reason
`superseded`	Context changed; result cannot be committed	Safe discard and reconciliation message

Guards: valid owner/session; writer lease; matching manuscript, tail page, canonical revision, and context fingerprint; unconsumed prepared identity where relevant.

Invariants: one provider owner; one canonical consumption; partial streams never canonical; known spend survives failure/supersession; repeated idempotency key does not create another charge.

Restart: `requested` may be safely claimed; an abandoned `running` attempt is reconciled to interrupted/failed unless a provider-specific durable result can be proven; `succeeded` may commit only after all context guards are revalidated.

03 Prepared page

Prepared prose is a one-slot speculative machine scoped to a manuscript.

```
absent -> preparing -> ready -> consumed
            |          |
            v          +-> invalidated
          failed
```

At most one `ready` prepared page exists. Its opaque identity, prose, model result, and context fingerprint travel together. `Next Page` accepts only that identity. Direction, canon, local template, Author Canon, active-tail, or relevant setting changes invalidate it.

Preparation can spend provider credit, but it cannot alter narrative canon or continuity. A retry is a new preparation attempt after the failed/stale one is closed. The UI must distinguish "no preparation," "still preparing," "ready," and "failed"; green never means "generate something else."

04 Writer lease

```
free -> held -> renewed -> released
          |
          +-> expired -> claimed by new session
```

One session owns manuscript mutation while a short heartbeat is current. Other sessions may read. A conflicting mutation receives the owning-session state and a reconciliation path, not a silent last-write-wins result.

Lease expiry makes the manuscript claimable; it does not roll back committed work and does not prove a provider attempt failed. Operation state and lease state are separate because the network may outlive a browser tab.

05 Page revision and canonical pointers

Revisions are immutable records, not mutable states. The page pointers form the state machine:

```
canonical = display
      |
      +-- active-tail substantive edit --> canonical' = display'
      |
      +-- historical copyedit ----------> canonical, display'
```

A substantive active edit invalidates preparation and creates continuity work for the new canonical revision. A historical copyedit changes visible/exported prose but leaves extracted memory tied to its original canonical evidence. The UI must show this distinction before save.

No transition overwrites revision text. Parents and kinds preserve lineage. A stale editor must fail its expected-revision guard.

06 Continuity delta

```
pending -> ready
   |
   +-> failed -> retry attempt -> ready | failed
```

One logical continuity work item belongs to one canonical page revision. `pending` means prose is valid but memory is incomplete. `ready` contains strict versioned structured output with page/revision provenance. `failed` stores error code, reason, model, and repair route.

Input priority for heavy manuscripts: Main Character perspective anchor first, then support cast, then background setting and background cast. A Main Character-driven story retains that anchor even if the page does not name the Main Character.

Guards: revision remains canonical for that page; configured Archivist is available; response satisfies JSON/schema/evidence rules.

Retry: page-local and explicit. It must not add a second ready delta or replay the full novel. Malformed JSON, schema mismatch, missing evidence, provider refusal, timeout, and context pressure remain distinct diagnoses.

07 Deterministic continuity fold

The fold is a projection, not a provider job:

```
ordered ready deltas + active corrections + Author Canon
                        |
                        v
                current remembered state
```

The same ordered evidence produces the same result. A missing/failed delta makes coverage incomplete; it is not treated as an empty delta. Derived checkpoints and search indexes can be deleted and rebuilt without losing truth.

A correction overlays extracted evidence without rewriting it. Author Canon supplies explicit author-owned facts/events. Precedence rules must be stable, documented, and tested with conflicting evidence.

08 Continuity issue

```
open -> acknowledged -> resolved
  |                       ^
  +---- evidence changes ---+
```

An issue is a derived warning connecting an author correction or changed fact to later prose that may now diverge. `acknowledged` means the author has seen it, not that prose is correct. `resolved` records the author's disposition/evidence.

Deterministic search creates candidates. Optional AI may summarize impact, but cannot transition the issue or edit prose. Rebuilding derived issues must preserve explicit author dispositions where identity/evidence still match.

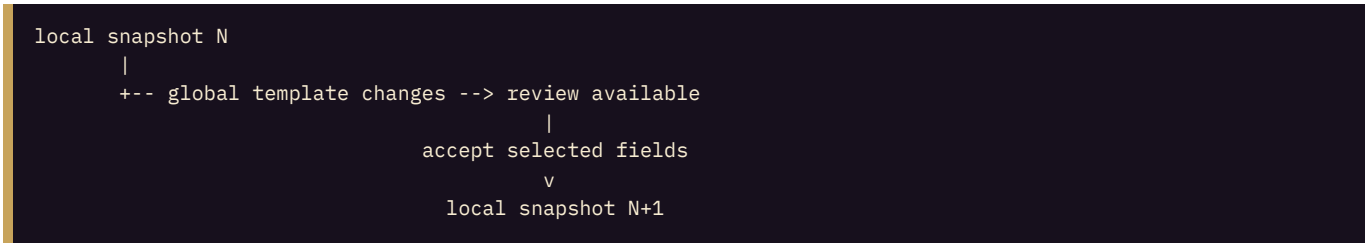
09 Author Canon record

```
active v1 -> active v2 -> active v3
    |                       |
    +------> retired <------+
                |
                +-> restored as new active version
```

Facts, world events, people, places, factions, objects, rules, and other record kinds are manuscript-local. Editing creates a new version. Retirement removes a record from active truth while preserving its history and citations. Restore creates another version; it does not erase the retirement event.

Author Canon is editable without AI. Any preparation whose context depended on a changed record is invalidated. Historical prose is never automatically rewritten.

10 Template snapshot review



Global Library edits do not mutate manuscript-local snapshots. Review computes a field-level diff. The author accepts selected fields explicitly. Applying a new snapshot invalidates relevant prepared work but does not rewrite prose or continuity.

Deleting or renaming the global source cannot orphan the local snapshot; generation reads the snapshot, not the live catalog row.

11 Recovery suffix

```
recoverable -> restored
      |
      +-> exported
      |
      +-> expired
```

Truncation first packages removed pages, revisions, continuity, directions, and private operation history, then updates canon transactionally. `recoverable` includes a surviving-head fingerprint and expiry.

Restore is allowed only while the current surviving head matches. `restored` reattaches the exact suffix once. `exported` preserves material for manual reconciliation when safe automatic restore is impossible. `expired` is no longer eligible for in-product restore.

Immediate undo is a privileged short path over the same evidence, not a separate manuscript model.

12 Asset processing

```
staging -> ready
   |
   +-> failed
```

Uploads and generated media share the technical boundary after bytes arrive: bounded stream, signature/decode verification, pixel limits, metadata stripping, normalized derivative, random storage name, atomic publish. `staging` is private and not renderable. `ready` alone may be placed or exported.

Failure cleans temporary files on request completion, timeout, cancellation, and startup reconciliation. Uploading never invokes a provider. AI generation has its own paid operation before media enters staging.

13 Image prompt refusal and sanitation

```
draft prompt -> provider refusal -> sanitized draft shown
                                          |
                                 explicit owner retry
                                          v
                                 generation operation
```

Sanitation does not continue generation. The rewritten prompt is visible, editable, and costed. The original prompt and upload remain unchanged. Only a new owner action creates the next paid operation.

This machine prevents a provider-specific interoperability step from becoming hidden moderation or hidden spend.

14 Asset placement

Placements are durable relationships, not asset states:

```
unplaced -> placed(anchor,page-order) -> moved -> unplaced
```

An anchor is before the first page or after a stable page ID. Page insertion/reorder does not corrupt it. Truncating a referenced page keeps the asset but removes or repairs the invalid placement. Multiple assets at one anchor have scoped order.

Placement never changes prose canon or continuity. Publication snapshots freeze the placement selected at creation time.

15 Narration cache and audiobook job

Page narration may be streamed from supported providers and cached by a hash of text, model, and voice. Cache replay must not create provider spend.

```
absent -> generating -> cached
            |
            +-> failed
```

Audiobooks are durable jobs:

```
pending -> running -> ready
             |
             +-> cancelled
             +-> failed
             +-> interrupted (restart)
```

Long text is segmented at safe sentence boundaries. Segment usage is aggregated once. PCM-only results are assembled into a valid WAV; MP3-capable results stream. `ready` requires a complete playable artifact, never a partial segment set.

16 Publication job

```
pending -> running -> ready
              |
              +-> failed
              +-> cancelled
              +-> interrupted
```

The job first freezes one `PublicationDocument`, then an adapter renders a format. The snapshot is immutable even if authoring continues. `ready` means the artifact validates sufficiently for download. Cancellation/failure cannot leave a file advertised as complete.

Restart marks in-memory-only `running` work interrupted unless durable adapter state has an explicit safe resume contract. Retrying creates a new job from a newly reviewed or deliberately reused snapshot.

17 Public share

```
active -> revoked
   |
   +-> expired (time guard)
```

An active share maps a stored hash of a high-entropy capability to one immutable publication snapshot. Reading does not refresh or mutate it. Revocation is irreversible for that capability; republishing creates a new token and optionally a new snapshot.

Unknown, expired, and revoked capabilities fail closed without revealing whether a private manuscript exists. Public reads offer no provider action or mutable private API.

18 Portable export

```
plan -> reviewed token -> streamed once -> consumed/expired
```

Planning enumerates scope, media/history choices, exposure, and exclusions without building an unreviewed archive. A reviewed token authorizes one bounded stream of the declared graph. Credentials, sessions, paid consent, recovery tokens, and share capabilities are impossible members.

A failed stream never changes catalogue state. Retry requires a new plan or an explicitly valid token according to the implementation contract.

19 Portable import

```
upload -> staged -> validated -> preflighted -> committed
   |         |          |             |
   +---------+----------+-------------+-> rejected/cleaned
```

Validation covers family/version, traversal, backslashes, symlinks, duplicates, undeclared entries, counts, expansion ratio, hashes, media decode, and safe IDs. Preflight presents collisions without catalogue mutation.

Commit stages sibling filesystem files, records rollback moves, and uses one SQLite transaction. Full replace first creates a persistent full safety archive. A crash cannot produce a committed database pointing at missing final media without reconciliation evidence.

20 Authentication session

```
unauthenticated -> setup | login -> authenticated -> locked/logged out/expired
```

Setup exists only before an owner is established. Successful authentication rotates opaque session identity. State-changing requests additionally require CSRF and same-origin/Host checks. Logout/revocation/expiry make replay fail.

A remembered browser session can reopen private local content after restart, but it cannot reconstruct an encrypted vault key. Authentication and provider-vault unlock are related but distinct machines.

21 Provider vault

```
unconfigured -> configured+locked -> unlocked -> locked
                       |             |
                       +-- reset ----+-> credentials cleared
```

Entry secrets are encrypted under a random data key. The owner passphrase derives a separate wrapping key. Password change rewraps the data key. Terminal recovery cannot obtain the old wrapping key and therefore clears stored provider credentials while preserving manuscripts.

The API may expose `locked`, `unavailable`, or repair states, but never plaintext. An environment-provided key is read-only configuration outside this vault machine.

22 Provider role assignment

```
unconfigured -> available
       |            |
       +-> unavailable/error/locked
```

Scribe, Archivist, and Narrator are logical roles. Discovery may report that a stored model is unavailable, but it cannot silently replace it. An explicit invalid `CONTINUITY_MODEL` refuses startup before listen. The user's browser-local Scribe selection must not redirect automatic Chronicle work.

23 Database startup

```
candidate -> identity proven -> migrated -> reconciled -> listening
    |
    +-> refused (3.x / unknown / future / checksum / corruption)
```

No schema write occurs before identity proof. Earlier 4.0 pre-rebrand adoption takes a complete SQLite snapshot before identity changes. Migration steps are ordered, checksum-proven, and transactional. Listening is the final state, not an optimistic early side effect.

In 4.1.0, schema 13 first proves or completes canonical page revisions and revision-bound continuity deltas, then retires the older writable mirrors. A page-shaped read view may preserve an API projection, but it is not another state owner. A failed proof rolls back the migration and prevents the server from listening.

24 UI request state

Every user-facing asynchronous surface should distinguish:

```
idle -> loading -> success | empty | failure | cancelled | stale
```

Failure includes a specific next action. Stale results must not paint into a newly selected manuscript. Dialogs and controls remain keyboard-operable, focus is restored, and danger/paid/provider states are not communicated by color alone.

The UI does not create durable truth by itself; it reflects server state and sends guarded commands. Refresh must reconstruct the same meaningful state.

25 Cross-machine invariants

Invariant	Machines that enforce it
Speculative prose never becomes truth accidentally	Writing operation, prepared page, canon pointers
One action cannot spend twice silently	Operation idempotency, provider result, UI request state
Memory cites the prose that produced it	Page revision, continuity delta, fold
Author truth outranks extracted suggestion	Author Canon, correction, fold, issue
Another tab cannot commit stale context	Writer lease, expected revision, operation supersession
Restart never invents success	Startup reconciliation, jobs, staging, operation journal
Public readers cannot reach mutable work	Publication snapshot, share capability
Transfer cannot escape its staging boundary	Import validation, filesystem commit/rollback
Media placement cannot renumber narrative truth	Asset placement, stable page IDs
Global templates cannot drift existing stories	Snapshot review, prepared invalidation

26 Test traceability checklist

For every transition, tests should cover happy path, invalid predecessor, repeated request, concurrent request, stale context, provider timeout, malformed result, cancellation, process restart, user-visible error, and persistence reload where applicable.

The highest-risk transitions require integration evidence across the real persistence boundary:

- `succeeded -> committed` for paid prose;
- `pending -> ready/failed` for memory;
- active-tail edit and historical copyedit pointer changes;
- truncation to recoverable suffix and exact restore;
- asset staging to atomic ready publication;
- import preflight to transactional commit/rollback;
- publication running to validated ready artifact; and
- active share to revoked/expired read denial.

An implementation that adds a status string without defining its transitions, restart behavior, and UI meaning has not added a state machine. It has added ambiguity.

27 5.0 release-branch story transitions

The playable-fiction foundation introduces one request path: `pending -> succeeded | failed | interrupted`. Success atomically records a new immutable beat, its complete state and the branch head. Malformed or stale replies record known spend without saving prose or partial effects. Reusing a successful key returns the earlier result without another provider call; a failed key never silently purchases a retry. Startup marks abandoned requests interrupted.

Branch selection, forks, control handoffs, corrections and episode transitions are local operations. They reject stale revisions and pending paid work. A fork references an exact ancestor rather than copying only its transcript. The default control state names no inhabited character. Explicitly taking or releasing a cast member creates a control beat; Ask creates clarification, not a fictional event.

An ended episode rejects continuation until the reader explicitly starts another. Reading, reloading or time spent away performs no narrative transition. No automatic world simulation punishes absence.

The reader controller adds route-generation and operation guards. Click -> busy precedes any consent dialog or network work. Cancel -> idle preserves the draft; success -> render clears only the submitted draft; failure -> free reconciliation preserves input. A route change invalidates late rendering but not server work. Lock invalidates every outstanding UI generation and clears private state. Pending server work is polled read-only; the browser never purchases a retry on a timer.